

STM32 DMA

直接内存访问 完全讲解

芯片平台	STM32F407 (Cortex-M4, 168MHz)
DMA 控制器	DMA1 / DMA2, 各 8 个 Stream, 共 16 路
HAL 库版本	STM32 HAL (CubeMX 生成框架)
内容涵盖	架构 / 配置 / UART / ADC / SPI / 内存 / 调试

2026-06-14

第一章 DMA 基本概念

1.1 什么是 DMA?

DMA (Direct Memory Access, 直接内存访问) 是一种硬件机制, 允许外设或内存之间直接传输数据, 无需 CPU 参与每一个字节的搬运。

传统方式 (CPU 轮询/中断) : 每个字节都要 CPU 执行「读寄存器-写内存」指令, CPU 忙于数据搬运而无法处理其他任务。

DMA 方式: CPU 只需配置好 DMA 参数 (源地址、目的地址、数据量) , 剩余工作由 DMA 控制器自主完成, CPU 可并行处理其他任务。传输结束后 DMA 触发中断通知 CPU。

1.2 CPU 搬运 vs DMA 搬运对比

对比项	CPU 轮询/中断搬运	DMA 搬运
CPU 占用	高, 每字节均需 CPU 参与	极低, 配置后 CPU 可做其他事
实时性	中断响应延迟积累	DMA 独立总线, 不占 CPU 流水线
吞吐量	受 CPU 指令速度限制	受总线带宽限制, 可达总线极限速率
适用场景	数据量少 (<几十字节)	数据量大 (ADC采样/串口收发/图像)
代码复杂度	简单	配置参数较多, 但 HAL 封装后易用

1.3 DMA 能做哪些方向的传输?

STM32 DMA 支持三种传输方向:

传输方向	典型场景	HAL 宏/方向常量
外设 → 内存 (Peripheral to Memory)	串口接收、ADC采样结果写入数组	DMA_PERIPH_TO_MEMORY
内存 → 外设 (Memory to Peripheral)	串口发送、SPI写屏幕/Flash	DMA_MEMORY_TO_PERIPH
内存 → 内存 (Memory to Memory)	memcpy加速、缓冲区复制	DMA_MEMORY_TO_MEMORY (仅DMA2支持)

□ 内存 → 内存 方向仅 DMA2 支持! DMA1 不支持 M2M 传输, 配置时务必选 DMA2。

第二章 STM32F407 DMA 架构

2.1 DMA1 与 DMA2 总线连接

STM32F407 有两个独立的 DMA 控制器，各挂接不同的 AHB 总线矩阵：

DMA1：连接到 AHB1，主要服务 APB1
外设（TIM2~7、USART2/3/4/5、I2C1/2/3、SPI2/3、DAC、I2S 等）。

DMA2：连接到 AHB1（数据）+ AHB2（外设），主要服务 APB2 及 AHB
外设（USART1/6、SPI1/4/5/6、TIM1/8、ADC1/2/3、SDIO、以太网、USB OTG、DCMI），同时支持 M2M（内存到内存）传输。

2.2 Stream / Channel 层级结构

每个 DMA 控制器有 8 个 Stream（数据流），每个 Stream 有 8 个 Channel（通道）可选择。Stream 是真正的传输通道，Channel 决定触发该 Stream 的外设请求源。

层级关系：

DMA 控制器 (DMA1 或 DMA2)
Stream 0 ~ Stream 7（8个独立传输流，可并行工作）
每个 Stream：Channel 0 ~ Channel 7（选择外设请求源）
每个 Stream 的核心寄存器：SxCR（控制）、SxNDTR（数量）、SxPAR（外设地址）、SxM0AR/SxM1AR（内存地址）、SxFCR（FIFO控制）

★ 选定 Stream 和 Channel 时必须查《STM32F4xx 参考手册》的 DMA Request Mapping 表，每个外设请求只能映射到固定的 Stream/Channel 组合，不可随意选择。

2.3 常用外设 DMA 请求映射（精选）

外设	DMA 控制器	Stream	Channel	方向
USART1_RX	DMA2	Stream 2 或 5	Channel 4	外设→内存
USART1_TX	DMA2	Stream 7	Channel 4	内存→外设
USART2_RX	DMA1	Stream 5	Channel 4	外设→内存
USART2_TX	DMA1	Stream 6	Channel 4	内存→外设
SPI1_RX	DMA2	Stream 0 或 2	Channel 3	外设→内存
SPI1_TX	DMA2	Stream 3 或 5	Channel 3	内存→外设
SPI2_RX	DMA1	Stream 3	Channel 0	外设→内存
SPI2_TX	DMA1	Stream 4	Channel 0	内存→外设

ADC1	DMA2	Stream 0 或 4	Channel 0	外设→内存
ADC2	DMA2	Stream 2 或 3	Channel 1	外设→内存
ADC3	DMA2	Stream 0 或 1	Channel 2	外设→内存
TIM1_UP	DMA2	Stream 5	Channel 6	内存→外设
TIM3_CH1	DMA1	Stream 4	Channel 5	内存→外设
I2C1_RX	DMA1	Stream 0 或 5	Channel 1	外设→内存
I2C1_TX	DMA1	Stream 1 或 6	Channel 1	内存→外设
M2M（任意）	DMA2	Stream 0~7	Channel 0	内存→内存

□ 同一 DMA 控制器中，两个 Stream 不能同时操作同一个外设的同一方向。CubeMX 会自动检测冲突，手动配置时需注意。

2.4 DMA 优先级仲裁

当多个 Stream 同时请求总线时，DMA 内部仲裁按以下规则决定顺序：

- 1. 软件优先级（DMA_SxCR.PL 字段）：Very High > High > Medium > Low，可在 HAL 中配置。
- 2. 软件优先级相同时：Stream 编号小的优先（Stream 0 > Stream 7）。
- 3. DMA2 与 DMA1 各自独立仲裁，两者访问总线时还受 AHB 总线仲裁影响。

```
// HAL 中设置优先级
hdma_usart1_rx.Init.Priority = DMA_PRIORITY_HIGH; // Very High
hdma_usart1_tx.Init.Priority = DMA_PRIORITY_MEDIUM;
hdma_adc1.Init.Priority = DMA_PRIORITY_LOW;
```

第三章 DMA 传输模式与 FIFO

3.1 三种传输模式

模式	描述	典型用途
正常模式 (Normal)	传输 NDTR 个数据后自动停止，产生传输完成中断（TC）。需重新调用 HAL 函数才能再次传输。	一次性数据块传输 (如发送固定长度帧)
循环模式 (Circular)	传输完成后 NDTR 自动重装，不需要 CPU 重启 DMA，可产生半传输（HT）和全传输（TC）中断。	ADC 连续采样 串口持续接收
双缓冲模式 (Double Buffer)	两个内存缓冲区交替使用：DMA 写 M0 时 CPU 处理 M1，DMA 写 M1 时 CPU 处理 M0，实现零拷贝乒乓操作。	高速数据流（SDIO/USB/DCMI） 图像帧缓冲

3.2 数据宽度与对齐

DMA 支持 3 种数据宽度：Byte（8位） / Half Word（16位） / Word（32位）。源和目的可以独立设置不同宽度，DMA 会自动做宽度转换（通过 FIFO 缓冲）。

```
// 数据宽度配置示例
hdma_adc1.Init.PeriphDataAlignment = DMA_PDATAALIGN_HALFWORD; // ADC 12位，用半字
hdma_adc1.Init.MemDataAlignment = DMA_MDATAALIGN_HALFWORD;

hdma_usart1_rx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE; // UART 8位
hdma_usart1_rx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE;
```

★ 若外设是 16 位 ADC、内存也是 uint16_t 数组，两端都配 HALFWORD；
若外设是 UART（8 位），两端都配 BYTE。宽度不匹配不报错但数据会对齐填充，需谨慎。

3.3 地址递增（Increment）设置

外设地址（PAR）通常固定不变（外设寄存器地址不变），内存地址（MAR）每次传输后自动递增。

```
hdma_usart1_rx.Init.PeriphInc = DMA_PINC_DISABLE; // 外设地址固定（DR寄存器）
hdma_usart1_rx.Init.MemInc = DMA_MINC_ENABLE; // 内存地址递增（填充数组）

// M2M 时两端都递增
hdma_m2m.Init.PeriphInc = DMA_PINC_ENABLE;
hdma_m2m.Init.MemInc = DMA_MINC_ENABLE;
```

3.4 FIFO 模式

STM32F4 DMA 每个 Stream 有 4 字（16 字节）的 FIFO 缓冲。

直接模式（FIFO 关闭）：每个外设请求立即触发一次 DMA 传输，延迟最低，适合 UART/SPI/I2C 等逐字节触发的外设。

FIFO 模式（开启）：数据先填入 FIFO 再批量写出，可实现宽度转换，适合 ADC/SDIO/DCMI 等需要数据宽度适配或突发传输的场景。

```
// 直接模式（UART 常用）
hdma_usart1_rx.Init.FIFOMode = DMA_FIFOMODE_DISABLE;

// FIFO 模式（ADC 多通道扫描时）
hdma_adc1.Init.FIFOMode = DMA_FIFOMODE_ENABLE;
hdma_adc1.Init.FIFOThreshold = DMA_FIFO_THRESHOLD_HALFFULL;
```

□ UART/SPI 使用直接模式（FIFO 关闭）；ADC 多通道扫描或需要宽度转换时才开启 FIFO。
FIFO 模式下不能使用 PBURST（外设突发），UART 等单字节外设不支持突发。

第四章 DMA 中断

4.1 DMA 的 5 种中断标志

中断标志	触发时机	常用场景
TCIF 传输完成	NDTR 减到 0，全部数据传输完成	正常模式：数据接收/发送完成通知
HTIF 半传输完成	NDTR 减到初始值一半（循环/双缓冲模式）	循环模式：乒乓处理半缓冲数据
TEIF 传输错误	总线错误或 FIFO 下溢/上溢导致传输中止	异常处理，打 log 或复位 DMA
DMEIF 直接模式错误	直接模式下 FIFO 发生错误	一般与 TEIF 一起处理
FEIF FIFO 错误	FIFO 溢出或下溢	开启 FIFO 模式时关注

4.2 HAL 中断回调函数

HAL 库中 DMA 中断通过以下弱函数（weak callback）通知应用层：

```
// 传输完成回调 (TCIF)
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    // UART DMA 接收完成
    if (huart->Instance == USART1) {
        process_rx_data(rx_buf, RX_BUF_SIZE);
        HAL_UART_Receive_DMA(&huart1, rx_buf, RX_BUF_SIZE); // 重启
    }
}

// 半传输完成回调 (HTIF) —— 循环模式乒乓处理
void HAL_UART_RxHalfCpltCallback(UART_HandleTypeDef *huart) {
    process_rx_data(rx_buf, RX_BUF_SIZE / 2); // 处理前半缓冲
}

// 传输错误回调
void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart) {
    HAL_UART_Receive_DMA(&huart1, rx_buf, RX_BUF_SIZE); // 复位并重启
}
```

4.3 NVIC 中断优先级配置

DMA Stream 中断需要在 NVIC 中使能并设置优先级。若与 FreeRTOS 配合使用，中断优先级数值必须大于等于 configMAX_SYSCALL_INTERRUPT_PRIORITY（通常为 5）。

```
// CubeMX 生成的典型配置
```



```
HAL_NVIC_SetPriority(DMA2_Stream2_IRQn, 5, 0); // USART1_RX DMA
HAL_NVIC_EnableIRQ(DMA2_Stream2_IRQn);

HAL_NVIC_SetPriority(DMA2_Stream7_IRQn, 5, 0); // USART1_TX DMA
HAL_NVIC_EnableIRQ(DMA2_Stream7_IRQn);
```

★ 与 FreeRTOS 同时使用时，所有 DMA 中断优先级数值必须 $\geq$ configMAX_SYSCALL_INTERRUPT_PRIORITY，否则在 DMA 回调中调用 FromISR API 会导致断言失败或系统崩溃。

第五章 UART + DMA 实战

5.1 DMA 发送 (TX)

UART DMA 发送是最简单的使用场景：将内存中的数据通过 DMA 自动搬运到 UART 数据寄存器 (DR/TDR)，发送完成后触发 TCIF 中断。

```
// 初始化 (CubeMX生成, 手动补充要点)
// 1. 在 MX_USART1_UART_Init() 后关联 DMA
// 2. HAL_UART_MspInit 中配置 DMA2 Stream7 Ch4 并使能 NVIC

// 发送数据 (非阻塞, DMA 自动完成)
uint8_t tx_buf[] = "Hello DMA!\r\n";
HAL_UART_Transmit_DMA(&huart1, tx_buf, sizeof(tx_buf));

// 等待发送完成 (若需要同步)
while (HAL_UART_GetState(&huart1) != HAL_UART_STATE_READY);

// 或者在回调中处理下一帧
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart->Instance == USART1) {
        // 发送完成, 可以准备下一包数据
    }
}
```

□ tx_buf 必须在 DMA 传输期间保持有效 (不能是局部变量在函数返回后被销毁) !
推荐定义为全局数组或静态变量。

5.2 DMA 接收 (RX) — 固定长度

适合协议帧长度固定 (如 Modbus RTU、自定义二进制帧) 的场景：

```
#define RX_LEN 8 // 协议帧固定 8 字节
uint8_t rx_buf[RX_LEN];

// 启动 DMA 接收 (正常模式, 收满 8 字节触发回调)
HAL_UART_Receive_DMA(&huart1, rx_buf, RX_LEN);

// 接收完成回调
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart->Instance == USART1) {
        parse_frame(rx_buf, RX_LEN); // 解析帧
        HAL_UART_Receive_DMA(&huart1, rx_buf, RX_LEN); // 重启接收
    }
}
```

```
}
```

5.3 DMA + 空闲中断 (IDLE) 接收不定长数据

最常用的 UART 接收方案：DMA 循环写入缓冲，总线空闲时（一帧数据结束后无新字节超过 1 字节时间）IDLE 中断触发，读取实际接收长度。

```
#define RX_BUF_SIZE 256

uint8_t rx_buf[RX_BUF_SIZE];

// 1. 使能 USART IDLE 中断 + 启动 DMA 接收
void uart_dma_idle_init(void) {
    __HAL_UART_ENABLE_IT(&huart1, UART_IT_IDLE);
    HAL_UART_Receive_DMA(&huart1, rx_buf, RX_BUF_SIZE);
}

// 2. USART1 中断服务函数 (stm32f4xx_it.c)
void USART1_IRQHandler(void) {
    HAL_UART_IRQHandler(&huart1);

    if (__HAL_UART_GET_FLAG(&huart1, UART_FLAG_IDLE)) {
        __HAL_UART_CLEAR_IDLEFLAG(&huart1); // 清除 IDLE 标志
        HAL_UART_DMAStop(&huart1); // 停止本次 DMA

        // 计算实际接收字节数 = 缓冲区大小 - DMA 剩余计数
        uint16_t recv_len = RX_BUF_SIZE
            - (uint16_t)__HAL_DMA_GET_COUNTER(huart1.hdmarx);

        process_rx_data(rx_buf, recv_len); // 业务处理

        HAL_UART_Receive_DMA(&huart1, rx_buf, RX_BUF_SIZE); // 重启
    }
}
```

★ IDLE 中断 + DMA 是嵌入式串口接收不定长数据的工程标准方案，CPU 占用极低，STM32 HAL 项目必掌握。

5.4 HAL_UART_Receive_DMA 内部做了什么？

HAL_UART_Receive_DMA 内部流程（简化）：

```
HAL_StatusTypeDef HAL_UART_Receive_DMA(UART_HandleTypeDef *huart,
                                         uint8_t *pData, uint16_t Size) {
    // 1. 配置 DMA：源=USART_DR，目的=pData，数量=Size
    huart->hdmarx->Init.Direction = DMA_PERIPH_TO_MEMORY;
    huart->hdmarx->Init.PeriphInc = DMA_PINC_DISABLE;
```

```
huart->hdmarx->Init.MemInc = DMA_MINC_ENABLE;
HAL_DMA_Init(huart->hdmarx);

// 2. 设置 DMA 传输完成、半完成、错误回调
huart->hdmarx->XferCpltCallback = UART_DMAReceiveCplt;
huart->hdmarx->XferHalfCpltCallback = UART_DMARxHalfCplt;
huart->hdmarx->XferErrorCallback = UART_DMAError;

// 3. 启动 DMA
HAL_DMA_Start_IT(huart->hdmarx,
                 (uint32_t)&huart->Instance->DR,
                 (uint32_t)pData, Size);

// 4. 使能 USART DMA 接收请求位
SET_BIT(huart->Instance->CR3, USART_CR3_DMAR);
return HAL_OK;
}
```

第六章 ADC + DMA 实战

6.1 为什么 ADC 要用 DMA?

ADC 多通道扫描模式 (Scan Mode) 会依次转换所有通道, 每次转换完成产生一个 EOC (End Of Conversion) 事件。若用中断方式, 每转换一个通道就进中断读取, CPU 负担极重。

使用 DMA: ADC 每完成一次转换, 自动触发 DMA 将结果写入内存数组, CPU 无需干预每一次采样, 等所有通道一轮扫描完成后 DMA 触发 TCIF 中断, CPU 批量处理。

6.2 ADC 多通道 DMA 连续采样配置

```
// 假设采集 4 个通道, 采样结果存入数组
#define ADC_CH_NUM 4

uint16_t adc_val[ADC_CH_NUM]; // 全局/静态! DMA 期间不能销毁

// --- ADC 初始化 (CubeMX 生成的关键部分) ---
hadc1.Init.ScanConvMode = ENABLE; // 扫描模式
hadc1.Init.ContinuousConvMode = ENABLE; // 连续转换
hadc1.Init.DMAContinuousRequests = ENABLE; // DMA 循环请求
hadc1.Init.EOCSelection = EOC_SEQ_CONV; // 序列结束产生 EOC
hadc1.Init.NbrOfConversion = ADC_CH_NUM;
HAL_ADC_Init(&hadc1);

// --- 配置各通道 (顺序对应 adc_val 数组下标) ---
ADC_ChannelConfTypeDef sConfig = {0};
sConfig.SamplingTime = ADC_SAMPLETIME_84CYCLES;

sConfig.Channel = ADC_CHANNEL_0; sConfig.Rank = 1;
HAL_ADC_ConfigChannel(&hadc1, &sConfig);
sConfig.Channel = ADC_CHANNEL_1; sConfig.Rank = 2;
HAL_ADC_ConfigChannel(&hadc1, &sConfig);
sConfig.Channel = ADC_CHANNEL_4; sConfig.Rank = 3;
HAL_ADC_ConfigChannel(&hadc1, &sConfig);
sConfig.Channel = ADC_CHANNEL_5; sConfig.Rank = 4;
HAL_ADC_ConfigChannel(&hadc1, &sConfig);

// --- 启动 DMA 采集 ---
HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_val, ADC_CH_NUM);

// --- 采集完成回调 (一轮 4 通道全部完成) ---
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc) {
```

```
if (hadc->Instance == ADC1) {  
    // adc_val[0]~[3] 即为 4 个通道的当前采样值  
    float v0 = adc_val[0] * 3.3f / 4096.0f; // 转换为电压  
}  
}
```

★ adc_val 数组必须声明为全局或静态变量，且不能存放在 CCM SRAM (0x10000000) ，
因为 DMA 无法访问 CCM SRAM！

6.3 ADC DMA 不更新的常见原因

问题现象	可能原因	解决方法
adc_val 一直是 0 或初始值	数组放在了 CCM SRAM	改为普通 SRAM （全局变量不加 __attribute__((section(".ccmram"))))
只更新一次，之后固定	DMA 正常模式 + ContinuousConvMode 未开启	开启 ContinuousConvMode 或在回调中重启 HAL_ADC_Start_DMA
多通道数据乱序	Rank 配置与数组下标不对应	确认每个 Channel 的 Rank 与 adc_val 数组下标一致
FIFO 错误中断	开启了 FIFO 但 FIFO 阈值配置不合适	ADC 通常用直接模式（FIFOMode=DISABLE），或正确配置阈值

第七章 SPI + DMA 实战

7.1 SPI DMA 全双工收发

SPI 是全双工协议，每次通信同时进行 TX 和 RX。HAL 提供 HAL_SPI_TransmitReceive_DMA 实现 DMA 全双工传输：

```
#define SPI_BUF_SIZE 16
uint8_t spi_tx_buf[SPI_BUF_SIZE] = {0x01, 0x02, /* ... */};
uint8_t spi_rx_buf[SPI_BUF_SIZE];

// 拉低片选
HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, GPIO_PIN_RESET);

// 启动全双工 DMA 传输
HAL_SPI_TransmitReceive_DMA(&hspi1, spi_tx_buf, spi_rx_buf, SPI_BUF_SIZE);

// 传输完成回调
void HAL_SPI_TxRxCpltCallback(SPI_HandleTypeDef *hspi) {
    if (hspi->Instance == SPI1) {
        HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, GPIO_PIN_SET); // 释放片选
        process_spi_response(spi_rx_buf, SPI_BUF_SIZE);
    }
}
```

7.2 SPI 只发不收 (仅 TX DMA)

驱动 LCD / W25Q Flash 写操作时只需发送，可仅用 TX DMA：

```
// 向 LCD 写一行像素数据 (320 个像素 x 2 字节 = 640 字节)
#define LCD_LINE_BYTES (320 * 2)
uint16_t line_buf[320]; // RGB565 像素数据

HAL_SPI_Transmit_DMA(&hspi1, (uint8_t*)line_buf, LCD_LINE_BYTES);

// 发送完成回调
void HAL_SPI_TxCpltCallback(SPI_HandleTypeDef *hspi) {
    if (hspi->Instance == SPI1) {
        lcd_cs_deselect(); // 拉高片选
    }
}
```

★ 用 DMA 驱动 LCD 刷屏速度比阻塞方式快 10 倍以上，
DMA 传输期间 CPU 可执行动画计算逻辑，实现流畅帧率。

第八章 内存到内存 DMA (M2M)

8.1 M2M 用途与限制

内存到内存传输：通过 DMA2 实现高速

memcpy，适合需要频繁拷贝大块数据的场景（帧缓冲、双缓冲区切换、SDRAM 操作）。

限制：只有 DMA2 支持 M2M（DMA1 不支持）；M2M 模式不能与循环模式（Circular）同时使用。

8.2 M2M DMA 实现

```
DMA_HandleTypeDef hdma_m2m;

void dma_m2m_init(void) {
    __HAL_RCC_DMA2_CLK_ENABLE();

    hdma_m2m.Instance = DMA2_Stream0;
    hdma_m2m.Init.Channel = DMA_CHANNEL_0;
    hdma_m2m.Init.Direction = DMA_MEMORY_TO_MEMORY;
    hdma_m2m.Init.PeriphInc = DMA_PINC_ENABLE; // 源地址递增
    hdma_m2m.Init.MemInc = DMA_MINC_ENABLE; // 目标地址递增
    hdma_m2m.Init.PeriphDataAlignment = DMA_PDATAALIGN_WORD; // 32位
    hdma_m2m.Init.MemDataAlignment = DMA_MDATAALIGN_WORD;
    hdma_m2m.Init.Mode = DMA_NORMAL; // M2M 不能用循环模式
    hdma_m2m.Init.Priority = DMA_PRIORITY_HIGH;
    hdma_m2m.Init.FIFOMode = DMA_FIFOMODE_ENABLE;
    hdma_m2m.Init.FIFOThreshold = DMA_FIFO_THRESHOLD_FULL;
    HAL_DMA_Init(&hdma_m2m);
}

// 使用：异步复制 4KB 数据（约 CPU 速度的 2~3 倍）
#define BUF_SIZE 4096
uint8_t src[BUF_SIZE], dst[BUF_SIZE];

void fast_memcpy_dma(void *dst, const void *src, uint32_t len) {
    HAL_DMA_Start_IT(&hdma_m2m,
        (uint32_t)src,
        (uint32_t)dst,
        len / 4); // len 单位：字节，传输以字为单位
}

void HAL_DMA_M2M_CpltCallback(void) {
```

```
// 拷贝完成  
}
```

□ M2M DMA 使用 Word (32位) 宽度时, len 必须是 4 的整数倍。
源和目标地址也必须 4 字节对齐, 否则产生总线错误 (HardFault)。

第九章 双缓冲模式 (Double Buffer)

9.1 双缓冲工作原理

双缓冲模式 (Double Buffer Mode) 使用 M0AR 和 M1AR 两个内存地址：

DMA 写 M0 时，CPU 处理 M1 中已完成的数据；

M0 写满后 (HTIF 或 TCIF) ， DMA 自动切换到 M1， CPU 处理 M0。

这样 DMA 与 CPU 真正并行，实现零等待的数据流处理。

时刻	DMA 正在操作	CPU 正在处理
T1	写入 M0 (Buffer 0)	处理 M1 (Buffer 1) 中的旧数据
T2 (M0满, 切换)	写入 M1 (Buffer 1)	处理 M0 (Buffer 0) 中的新数据
T3 (M1满, 切换)	写入 M0 (Buffer 0)	处理 M1 (Buffer 1) 中的新数据

9.2 双缓冲配置示例 (USART RX)

```
#define BUF_SIZE 128

uint8_t buf0[BUF_SIZE], buf1[BUF_SIZE];

// 配置双缓冲模式
hdma_usart1_rx.Init.Mode = DMA_CIRCULAR; // 双缓冲依赖循环模式
HAL_DMA_Init(&hdma_usart1_rx);

// 使能双缓冲并设置两个内存地址
HAL_DMAEx_MultiBufferStart_IT(
    &hdma_usart1_rx,
    (uint32_t)&USART1->DR, // 外设地址
    (uint32_t)buf0, // M0AR: 第一个缓冲区
    (uint32_t)buf1, // M1AR: 第二个缓冲区
    BUF_SIZE
);

// 传输完成回调 (两个缓冲区满时交替触发)
void HAL_DMA_M2M_CpltCallback(DMA_HandleTypeDef *hdma) {
    // 判断当前 DMA 正在写哪个缓冲区
    if ((DMA2_Stream2->CR & DMA_SxCR_CT) == 0) {
        // CT=0: DMA 当前写 M0, CPU 处理 M1
        process_data(buf1, BUF_SIZE);
    }
}
```

```
} else {  
    // CT=1: DMA 当前写 M1, CPU 处理 M0  
    process_data(buf0, BUF_SIZE);  
}  
}
```

★ 双缓冲模式适用于高速连续数据流（SDIO 读卡、DCMI 摄像头、ADC 高速采样）。
UART 不定长帧一般用 IDLE+DMA 即可，双缓冲适合固定帧长的高速场景。

第十章 常见问题与调试

10.1 DMA 常见故障排查表

故障现象	根本原因	解决方法
DMA 完全不工作	外设 DMA 请求位未使能 (CR3.DMAR / CR2.TXDMAEN 等)	检查外设控制寄存器中 DMA 使能位; HAL 中通过 HAL_xxx_xxx_DMA 自动设置
数据只传一次, 之后不更新	正常模式未重启; 或 DMA 传输错误后未复位	在 TxCplt/RxCplt 回调中重新调用 HAL_xxx_Start_DMA 或 HAL_xxx_Receive_DMA
接收到数据但内容乱序	Stream/Channel 选择与外设不对应 (查错 Request Map)	对照参考手册 DMA Request Map 重新确认 Stream 和 Channel
HardFault 异常	DMA 目标缓冲区位于 CCM SRAM (DMA 不可达)	将缓冲区改为普通 SRAM; 不要用 .ccmram section 修饰 DMA 缓冲区
ADC DMA 采样值偏移或全为0	未开启 DMA 连续请求 (DMAContinuousRequests=DISABLE)	hadc.Init.DMAContinuousRequests = ENABLE
FIFO 错误中断频繁触发	开启 FIFO 但宽度配置不匹配 (阈值/数据宽度不对齐)	UART/SPI 改用直接模式 (FIFOMode=DISABLE)
DMA 传输异常停止	总线仲裁冲突或 过度访问AHB总线	降低 DMA 请求频率; 多个 DMA 时适当配置优先级

10.2 关键调试技巧

技巧 1 — 用 DMA 错误中断定位问题:

```
// 使能所有 DMA 错误中断, 出错时进入回调
void HAL_DMA_ErrorCallback(DMA_HandleTypeDef *hdma) {
    uint32_t err = HAL_DMA_GetError(hdma);
    // err 可能是以下位组合:
    // HAL_DMA_ERROR_TE -- 传输错误
    // HAL_DMA_ERROR_FE -- FIFO 错误
    // HAL_DMA_ERROR_DME -- 直接模式错误
    while (1); // 先死机, 便于 debugger 查看 err 值
}
```

技巧 2 — 用逻辑分析仪验证时序:

对于 UART/SPI DMA 传输, 用逻辑分析仪抓取总线波形, 确认数据内容和时序是否符合预期, 排除是数据源问题还是 DMA 配置问题。

技巧 3 — 先用阻塞方式验证外设, 再切换 DMA:

调试新外设时先用 HAL_UART_Receive /

HAL_SPI_TransmitReceive（阻塞方式）确认外设和引脚工作正常，再换成 DMA 方式，缩小排查范围。

10.3 DMA 与 Cache 一致性 (Cortex-M7 适用)

STM32F407 (Cortex-M4) 无数据 Cache，不存在 Cache 一致性问题。

STM32H7 / STM32F7 (Cortex-M7) 有 D-Cache：DMA 直接访问内存，若 Cache 未与内存同步，CPU 读到的是 Cache 中的旧数据。

```
// STM32H7 DMA 接收前/后的 Cache 操作 (SCB 函数)

// 接收前：将对应地址的 Cache 行无效化 (清除旧 Cache)
SCB_InvalidateDCache_by_Addr((uint32_t*)rx_buf, sizeof(rx_buf));
HAL_UART_Receive_DMA(&huart1, rx_buf, sizeof(rx_buf));

// 发送前：将内存数据刷入 Cache (确保 DMA 读到最新数据)
SCB_CleanDCache_by_Addr((uint32_t*)tx_buf, sizeof(tx_buf));
HAL_UART_Transmit_DMA(&huart1, tx_buf, sizeof(tx_buf));
```

□ STM32F407 无需 Cache 操作，但如果将来移植到 H7/F7，
必须在 DMA 操作前后加 Cache 维护指令，否则数据静默错误极难排查！

第十一章 完整工程示例——UART DMA + FreeRTOS

11.1 架构说明

典型的 STM32 + FreeRTOS 工程中 UART DMA 接收架构：

硬件层	USART1 (PA9=TX, PA10=RX)
DMA 层	DMA2 Stream2 Ch4 (RX) + DMA2 Stream7 Ch4 (TX)
中断层	USART1_IRQHandler → IDLE 检测帧结束 DMA2_Stream2_IRQHandler → 传输完成/错误
OS 层	IDLE 中断 → xQueueSendFromISR → UART 处理任务 UART 处理任务 → 解析帧 → 业务队列

11.2 关键代码

```
// uart_driver.h
#define UART_RX_BUF_SIZE 256
typedef struct {
    uint8_t data[UART_RX_BUF_SIZE];
    uint16_t len;
} UartFrame_t;

// uart_driver.c
static uint8_t s_rx_buf[UART_RX_BUF_SIZE];
static QueueHandle_t s_frame_queue;

void uart_dma_init(void) {
    s_frame_queue = xQueueCreate(8, sizeof(UartFrame_t));
    __HAL_UART_ENABLE_IT(&huart1, UART_IT_IDLE);
    HAL_UART_Receive_DMA(&huart1, s_rx_buf, UART_RX_BUF_SIZE);
}

// USART1 中断服务 (stm32f4xx_it.c 中调用本函数)
void uart_idle_irq_handler(void) {
    if (!__HAL_UART_GET_FLAG(&huart1, UART_FLAG_IDLE)) return;
    __HAL_UART_CLEAR_IDLEFLAG(&huart1);
    HAL_UART_DMAStop(&huart1);

    UartFrame_t frame;
    frame.len = UART_RX_BUF_SIZE
        - (uint16_t)__HAL_DMA_GET_COUNTER(huart1.hdmarx);
    memcpy(frame.data, s_rx_buf, frame.len);
}
```

```
BaseType_t woken = pdFALSE;
xQueueSendFromISR(s_frame_queue, &frame, &woken);
HAL_UART_Receive_DMA(&huart1, s_rx_buf, UART_RX_BUF_SIZE);
portYIELD_FROM_ISR(woken);
}

// FreeRTOS 处理任务
void uart_task(void *arg) {
    UartFrame_t frame;
    for (;;) {
        if (xQueueReceive(s_frame_queue, &frame, portMAX_DELAY)) {
            // 在任务上下文中安全处理帧数据
            parse_protocol(frame.data, frame.len);
        }
    }
}
```

★ memcpy(frame.data, s_rx_buf, frame.len) 在中断中将数据复制到队列帧，使 DMA 缓冲区立即可以被下一帧数据复用，避免数据被覆盖。